



FACADE PATTERN

Simplified Interface / API Gateway

1. Mục tiêu bài học

Sau bài này, sinh viên có thể:

- Hiểu bản chất Facade (Che giấu độ phức tạp)
- Nhận diện hệ thống con (Subsystems) rắc rối
- Thiết kế Facade cho Order System
- Phân biệt Facade vs API Gateway vs Proxy
- Kết hợp với Strategy, Factory, CoR

2. Bối cảnh – Hệ thống phình to

Hệ thống Order thực tế gồm nhiều Subsystems:

1. InventoryService (Check kho)
2. PricingService (Tính giá)
3. DiscountService (Giảm giá)
4. PaymentService (Thanh toán)
5. ShippingService (Vận chuyển)
6. NotificationService (Gửi mail)

❓ Nếu Client (UI/Mobile) phải gọi từng cái một:

- > Code Client phình to.
- > Client biết quá nhiều (Tight Coupling).

3. Thiết kế ban đầu – KHÔNG FACADE

✗ Vi phạm SRP, DIP và Tight Coupling nghiêm trọng.

```
public class OrderController {  
    public void Checkout(Order order) {  
        // Controller ôm đồm quá nhiều  
        _inventoryService.Check(order);  
        _discountService.Apply(order);  
        _paymentService.Pay(order);  
        _shippingService.Create(order);  
        _notificationService.Send(order);  
    }  
}
```

4. Tại sao thiết kế trên tồi?

Vấn đề	Giải thích
Tight Coupling	Controller biết tên và cách dùng của 5 services.
Khó thay đổi	Đổi thứ tự Payment/Shipping -> Sửa UI code.
Khó Reuse	Logic 'Checkout' bị dính chặt vào Controller.
Vi phạm DIP	Tầng cao (UI) phụ thuộc trực tiếp tầng thấp.

5. Ý tưởng Facade Pattern

👉 Ý tưởng cốt lõi:

Tạo một "Mặt tiền" (Facade) đứng ra nhận yêu cầu.

Client: Chỉ gọi 1 lệnh PlaceOrder().

Facade: Tự biết gọi Inventory -> Payment -> Shipping...

🔑 Nguyên tắc: "Talk to only one friend" (Least Knowledge).

6. Implementation – The Facade

✂ Facade chỉ điều phối, không nên chứa Business Logic phức tạp.

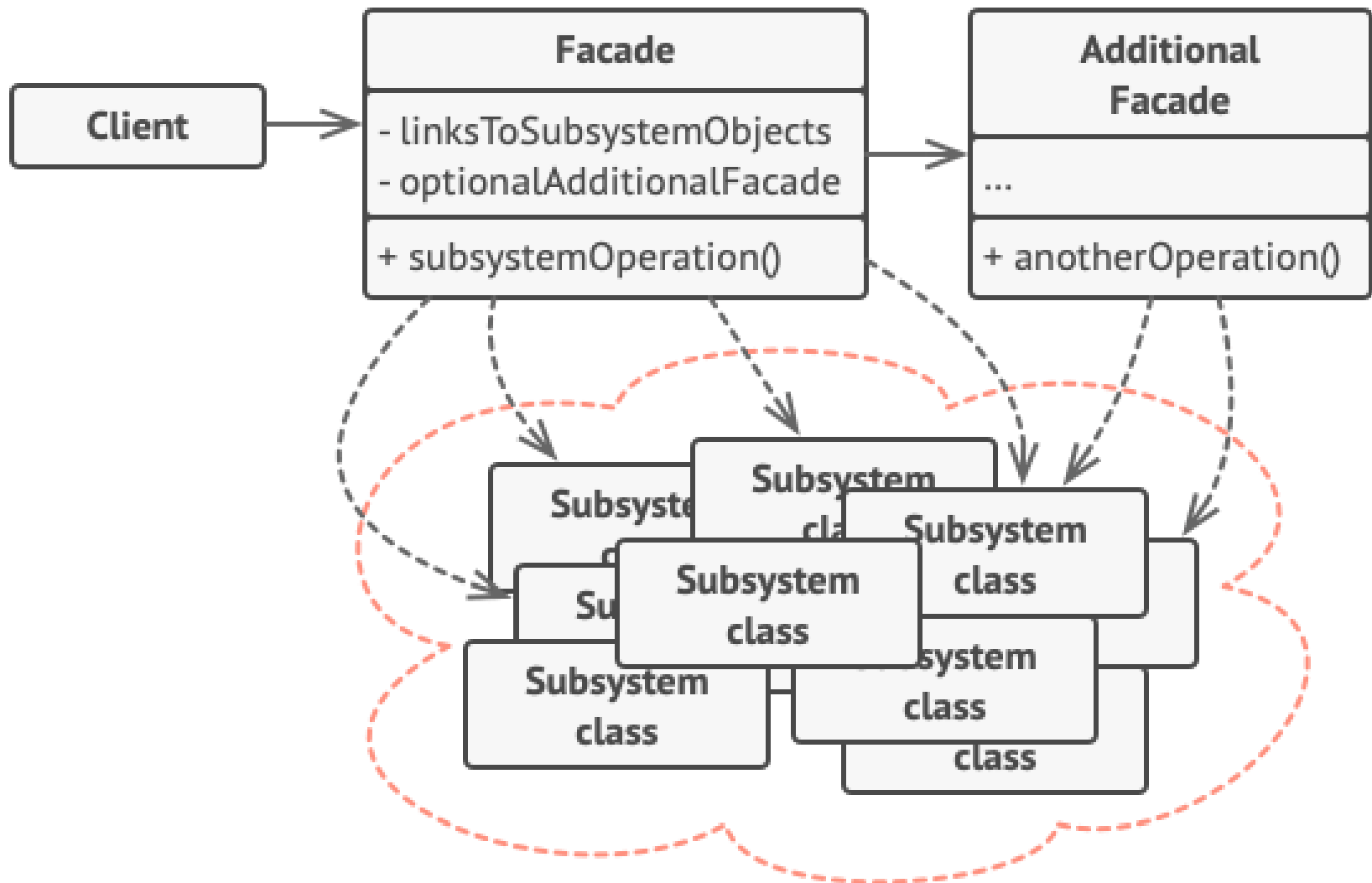
```
public class OrderFacade : IOrderFacade {
    // Inject tất cả Subsystems vào đây
    public OrderFacade(InventoryService inv, PaymentService pay,
    ...) { ... }

    public void PlaceOrder(Order order) {
        // Facade điều phối các bước
        _inventory.Check(order);
        _discount.Apply(order);
        _payment.Pay(order);
        _shipping.Create(order);
        _notification.Send(order);
    }
}
```

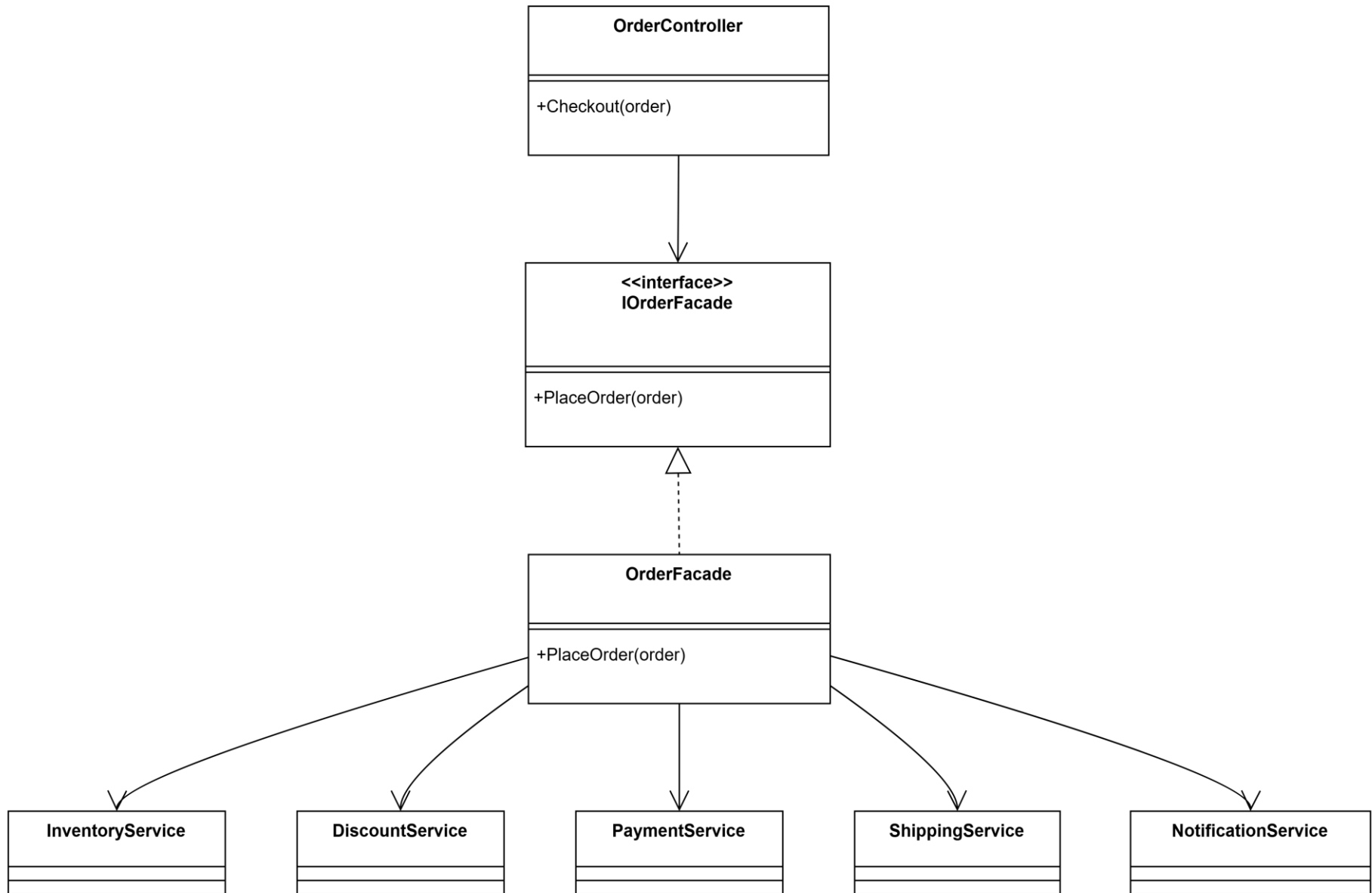
7. Client sau khi có Facade

```
public class OrderController {  
    private readonly IOrderFacade _orderFacade;  
  
    public OrderController(IOrderFacade orderFacade) {  
        _orderFacade = orderFacade;  
    }  
  
    public void Checkout(Order order) {  
        // Client cực kỳ đơn giản  
        _orderFacade.PlaceOrder(order);  
    }  
}
```


8. Structure of Facade Pattern



8. Facade Pattern: Case Study



9. Facade vs Các Pattern khác

Pattern	Mục đích
Facade	Đơn giản hóa Interface (Dễ dùng)
Adapter	Chuyển đổi Interface (Tương thích)
Strategy	Thay đổi thuật toán (Linh hoạt)
CoR	Xử lý theo chuỗi (Pipeline)
Proxy	Kiểm soát truy cập (Bảo mật/Lazy)

10. Facade trong Clean Architecture

UI / API (Controller)



Application Layer (Facade / Use Cases)



Domain Services (Business Logic)



Infrastructure (Subsystems)

📌 Facade thường nằm ở Application Layer.

📌 Facade là tiền thân của API Gateway trong Microservices.

11. Bài tập thực hành

Bài 1:

Thiết kế 'OrderCancellationFacade' xử lý các bước:
Refund -> Restock Inventory -> Notify Customer.

Bài 2:

So sánh OrderFacade (điều phối) vs OrderPipeline (Chain of Responsibility).
Khi nào dùng cái nào?

Bài 3:

Vẽ UML Diagram cho hệ thống SmartHomeFacade (Bật đèn, mở rèm, bật nhạc, chỉnh điều hòa).

12. Kết luận

“ Client càng đơn giản – Hệ thống càng bền. ”

“ Facade không làm giảm độ phức tạp của hệ thống, nó chỉ giấu độ phức tạp đó đi khỏi mắt người dùng. ”